

Latihan dan Tugas CRUD dengan PHP dan MySQL

Petunjuk

Agar dapat memahami dengan baik, lakukan langkah demi langkah sebagai berikut:

- Pertama buat CRUD tanpa pengamanan data, simpan dalam direktori crud1
- Baca bagian penjelasan penggunaan `mysql_real_escape_string()` lalu buat CRUD dengan `mysql_real_escape_string()`, simpan dalam direktori crud2
- Baca bagian penjelasan prepare statement lalu buat CRUD dengan prepare statement, simpan dalam direktori crud3

Pelajari perbedaan perintah dari ketiga versi CRUD tersebut.

CRUD tanpa pengamanan data

CRUD (Create, Read, Update, Delete) adalah operasi dasar dalam pengelolaan database. Dalam tutorial ini, kita akan membuat aplikasi CRUD sederhana menggunakan PHP dan MySQL.

Persiapan

Pastikan XAMPP terinstal dan Apache serta MySQL berjalan.

Buat database:

```
CREATE DATABASE crud_db;  
USE crud_db;
```

Buat tabel **users** dan **products**:

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  passw VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nama_produk VARCHAR(100) NOT NULL,  
  harga DECIMAL(10,2) NOT NULL,  
  stok INT NOT NULL  
);
```

Koneksi ke Database

Buat file **config.php** untuk menyimpan koneksi database:

```
<?php
$host = 'localhost';
$user = 'root';
$pass = '';
$dbname = 'crud_db';

$conn = new mysqli($host, $user, $pass, $dbname);

if ($conn->connect_error) {
    die("Koneksi gagal: " . $conn->connect_error);
}
?>
```

Membuat Operasi CRUD

a. Read Data (Menampilkan Data)

Buat file **index.php** untuk menampilkan data:

```
<?php
include 'config.php';
$result = $conn->query("SELECT * FROM users");
?>
<!DOCTYPE html>
<html>
<head>
    <title>CRUD PHP MySQL</title>
</head>
<body>
    <h2>Daftar Pengguna</h2>
    <a href="create.php">Tambah Pengguna</a>
    <table border="1">
        <tr>
            <th>ID</th><th>Nama</th><th>Email</th>
            <th>Password</th><th>Aksi</th>
        </tr>
        <?php while ($row = $result->fetch_assoc()): ?>
            <tr>
                <td><?= $row['id'] ?></td>
                <td><?= $row['name'] ?></td>
                <td><?= $row['email'] ?></td>
                <td><?= $row['passw'] ?></td>
                <td>
                    <a href="edit.php?id=<?= $row['id'] ?>">Edit</a> |
```

```

        <a href="delete.php?id=<?= $row['id'] ?>" onclick="return
confirm('Hapus data?')">Hapus</a>
    </td>
</tr>
<?php endwhile; ?>
</table>
</body>
</html>

```

b. Create Data (Menambah Data)

Buat file **create.php**:

```

<?php
include 'config.php';
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $email = $_POST['email'];
    $passw = $_POST['passw'];
    $conn->query("INSERT INTO users (name, email, passw) VALUES ('$name',
'$email', '$passw')");
    header("Location: index.php");
}
?>
<!DOCTYPE html>
<html>
<body>
    <h2>Tambah Pengguna</h2>
    <form method="POST">
        Nama: <input type="text" name="name" required><br>
        Email: <input type="email" name="email" required><br>
        Password: <input type="password" name="passw" required><br>
        <button type="submit">Simpan</button>
    </form>
</body>
</html>

```

c. Update Data (Mengedit Data)

Buat file **edit.php**:

```

<?php
include 'config.php';
$id = $_GET['id'];
$user = $conn->query("SELECT * FROM users WHERE id=$id")-
>fetch_assoc();
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];

```

```

$email = $_POST['email'];
$passw = $_POST['passw'];
$conn->query("UPDATE users SET name='$name', email='$email',
passw='$passw' WHERE id=$id");
header("Location: index.php");
}
?>
<!DOCTYPE html>
<html>
<body>
    <h2>Edit Pengguna</h2>
    <form method="POST">
        Nama: <input type="text" name="name" value="<?= $user['name'] ?>"
required><br>
        Email: <input type="email" name="email" value="<?= $user['email'] ?>"
required><br>
        Umur: <input type="password" name="passw" value="<?= $user['passw']
?>" required><br>
        <button type="submit">Update</button>
    </form>
</body>
</html>

```

d. Delete Data (Menghapus Data)

Buat file **delete.php**:

```

<?php
include 'config.php';
$id = $_GET['id'];
$conn->query("DELETE FROM users WHERE id=$id");
header("Location: index.php");
?>

```

Kesimpulan

Tutorial ini menjelaskan cara membuat CRUD sederhana menggunakan PHP dan MySQL. Dengan memahami dasar-dasar CRUD, kita dapat mengembangkan aplikasi web yang lebih kompleks dengan fitur tambahan seperti validasi input dan autentikasi pengguna.

Poin Penting!

Berikut adalah poin-poin penting yang perlu dipahami dalam CRUD dengan PHP dan MySQL:

1. Konsep CRUD

- **Create** → Menambahkan data baru ke database.
- **Read** → Menampilkan data yang tersimpan di database.
- **Update** → Mengubah data yang sudah ada.
- **Delete** → Menghapus data dari database.

2. Struktur Database dan Tabel

- Kita harus memahami bagaimana mendesain database dengan tabel yang memiliki primary key.
- Penggunaan tipe data yang sesuai (VARCHAR, INT, AUTO_INCREMENT, dsb.)

3. Koneksi ke Database

- Menggunakan mysqli atau PDO untuk menghubungkan PHP dengan MySQL.
- Menangani koneksi database dengan baik, termasuk menangani kesalahan koneksi.

4. Menjalankan Query SQL dalam PHP

- Mahasiswa perlu memahami bagaimana INSERT, SELECT, UPDATE, dan DELETE dieksekusi dalam PHP.
- Menggunakan mysqli_query() untuk menjalankan perintah SQL.

5. Form Handling dalam PHP

- Menggunakan \$_POST untuk menangkap input dari form pada operasi Create dan Update.
- Menggunakan \$_GET untuk mengambil parameter dari URL pada operasi Read dan Delete.

6. Validasi Input

- Memastikan data yang dikirim ke database valid dan aman.
- Menggunakan fungsi seperti mysqli_real_escape_string() untuk mencegah SQL Injection.

7. Mengelola Data Hasil Query

- Menggunakan mysqli_fetch_assoc() untuk menampilkan data dalam bentuk array asosiatif.
- Menampilkan data dengan HTML dalam bentuk tabel agar mudah dibaca.

8. Navigasi dan Penggunaan Link

- Menggunakan `<a href="edit.php?id=1">Edit</a>` untuk navigasi antar halaman CRUD.
- Memahami bagaimana parameter dikirim melalui URL untuk operasi Update dan Delete.

9. Menggunakan Konfirmasi dalam Delete

- Menggunakan `confirm("Apakah Anda yakin ingin menghapus?")` pada tombol hapus untuk menghindari penghapusan tidak sengaja.

10. Redirect Setelah Operasi CRUD

- Menggunakan `header("Location: index.php")` agar halaman kembali ke daftar setelah operasi Create, Update, atau Delete selesai.

MENGAMANKAN DATA INPUTAN

1. MySQLi Prepared Statements: Pengertian & Implementasi

Apa itu MySQLi Prepared Statements?

MySQLi **Prepared Statements** adalah cara aman untuk menjalankan perintah SQL di PHP dengan mencegah **SQL Injection**.

✦ Kenapa Menggunakan Prepared Statements?

- ✓ **Keamanan lebih baik** – Mencegah **SQL Injection**.
- ✓ **Efisiensi** – Query bisa dieksekusi berulang tanpa dikompilasi ulang.
- ✓ **Pemrosesan lebih cepat** – MySQL hanya melakukan parsing sekali.

💡 Cara Kerja MySQLi Prepared Statements

1. **Persiapkan query SQL dengan placeholder (?)**.
2. **Bind (ikat) parameter** (menghubungkan variabel PHP dengan query).
3. **Eksekusi query** dengan data yang diberikan.
4. **Ambil hasil (jika SELECT query)**.
5. **Tutup statement**.

✂ Contoh Implementasi MySQLi Prepared Statements

✓ 1. INSERT Data (Create)

```
<?php
include 'koneksi.php';

// Query dengan placeholder (?)
$stmt = $conn->prepare("INSERT INTO users (nama, email, password) VALUES
(?, ?, ?)");

// Bind parameter (s = string)
$stmt->bind_param("sss", $nama, $email, $password);

// Data yang akan dimasukkan
$nama = "John Doe";
$email = "john@example.com";
$password = password_hash("123456", PASSWORD_DEFAULT);

// Eksekusi statement
$stmt->execute();

echo "Data berhasil ditambahkan!";
```

```
// Tutup statement
$stmt->close();
$conn->close();
?>
```

💡 Penjelasan:

- `prepare()` → Menyiapkan query SQL dengan `?` sebagai placeholder.
 - `bind_param("sss", $nama, $email, $password)` → Mengikat variabel PHP ke SQL:
 - **s** = string
 - **i** = integer
 - **d** = double
 - `password_hash()` → Hashing password agar lebih aman.
-

✅ 2. SELECT Data (Read)

```
<?php
include 'koneksi.php';

// Query dengan placeholder
$stmt = $conn->prepare("SELECT id, nama, email FROM users WHERE email = ?");
$stmt->bind_param("s", $email);

// Data yang akan dicari
$email = "john@example.com";

// Eksekusi query
$stmt->execute();

// Ambil hasil
$result = $stmt->get_result();

// Tampilkan data
while ($row = $result->fetch_assoc()) {
    echo $row['id'] . " - " . $row['nama'] . " - " . $row['email'] . "<br>";
}

// Tutup statement
$stmt->close();
$conn->close();
?>
```


💡 Keuntungan:

- Lebih **aman**, tidak bisa dimanipulasi lewat input pengguna.
 - **Efisien**, bisa dipakai berkali-kali dengan data berbeda.
-

✅ 3. UPDATE Data

```
<?php
include 'koneksi.php';

// Query dengan placeholder
$stmt = $conn->prepare("UPDATE users SET nama=? WHERE email=?");
$stmt->bind_param("ss", $nama_baru, $email);

// Data yang akan diperbarui
$nama_baru = "Jane Doe";
$email = "john@example.com";

// Eksekusi
$stmt->execute();

echo "Data berhasil diperbarui!";

// Tutup statement
$stmt->close();
$conn->close();
?>
```

💡 Manfaat:

- Query lebih **rapi** dan **modular**.
 - Bisa digunakan untuk memperbarui banyak data tanpa menulis query berulang.
-

✅ 4. DELETE Data

```
<?php
include 'koneksi.php';

// Query dengan placeholder
$stmt = $conn->prepare("DELETE FROM users WHERE email=?");
$stmt->bind_param("s", $email);
```

```
// Data yang akan dihapus  
$email = "john@example.com";
```

```
// Eksekusi  
$stmt->execute();
```

```
echo "Data berhasil dihapus!";
```

```
// Tutup statement  
$stmt->close();  
$conn->close();  
?>
```

💡 Keunggulan:

- Mencegah **SQL Injection**, misalnya pengguna mencoba DELETE FROM users lewat form input.

🚀 Kesimpulan

- ♦ **MySQLi Prepared Statements** lebih aman, efisien, dan mudah digunakan dibanding query biasa.
- ♦ **Mencegah SQL Injection** dengan memisahkan data dari perintah SQL.
- ♦ **Dapat digunakan untuk semua operasi CRUD (Create, Read, Update, Delete).**

Dengan MySQLi Prepared Statements, **database aplikasi web lebih aman dan profesional!** 🎯

2. Mysqli_real_escape_string()

Tanpa mysqli_real_escape_string() (Rentan terhadap SQL Injection)

Jika kita tidak menggunakan mysqli_real_escape_string(), seorang pengguna jahat dapat memasukkan kode berbahaya dalam input form.

Misalnya, pengguna mengisi form dengan:

Nama: John

Email: test@example.com

Umur: 25'); DROP TABLE users; --

Tanpa proteksi, query yang dieksekusi bisa menjadi seperti ini:

```
INSERT INTO users (name, email, age) VALUES ('John',  
'test@example.com', '25'); DROP TABLE users; --')
```

Ini bisa menyebabkan **seluruh tabel users terhapus!**

Dengan mysqli_real_escape_string() (Lebih Aman)

Kita bisa melindungi aplikasi dengan menggunakan mysqli_real_escape_string() untuk **membersihkan input** sebelum dimasukkan ke database.

Contoh kode yang aman di **create.php**:

```
<?php  
include 'config.php';  
  
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $name = mysqli_real_escape_string($conn, $_POST['name']);  
    $email = mysqli_real_escape_string($conn, $_POST['email']);  
    $passw = mysqli_real_escape_string($conn, $_POST['password']);  
    $passw = password_hash($passw, PASSWORD_DEFAULT);  
  
    $query = "INSERT INTO users (name, email, passw) VALUES ('$name', '$email',  
'$passw')";  
    if ($conn->query($query)) {  
        header("Location: index.php");  
    } else {  
        echo "Gagal menambahkan data: " . $conn->error;  
    }  
}  
?>  
  
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>Tambah Pengguna</h2>
<form method="POST">
  Nama: <input type="text" name="name" required><br>
  Email: <input type="email" name="email" required><br>
  Umur: <input type="password" name="passw" required><br>
  <button type="submit">Simpan</button>
</form>
</body>
</html>
```

Penjelasan Keamanan

- **mysqli_real_escape_string()** akan **menghapus karakter berbahaya** seperti ' (kutip) dan -- sehingga input tidak dapat merusak query SQL.
- **(int) \$_POST['age']** memastikan bahwa umur selalu berupa angka, mencegah input teks yang tidak valid.
- **Validasi pada Form** (seperti required) membantu mencegah input kosong atau tidak sesuai format.

Dengan teknik ini, kita bisa membuat aplikasi CRUD yang **lebih aman dari serangan SQL Injection!** 🛡️ 🔥

TUGAS

Tugas 1 : Proyek Mini Aplikasi CRUD (Studi Kasus: Sistem Manajemen Inventaris Sederhana)